

People Counter

Edge Computing People Flow Analysis via Thermal Imaging

Petru Daisa

May 2026

Contents

1	Introduction	2
1.1	Context	2
1.2	Specific Domain	2
2	Constraints and Requirements	2
3	Analysis and Theoretical Foundation	3
3.1	System Overview	3
3.2	Detection	3
3.2.1	Background Estimation	3
3.2.2	Thresholding	3
3.2.3	Connected Component Labelling	3
3.2.4	Intensity-Weighted Centroid	4
3.2.5	Multi-Person Blob Splitting via PCA	4
3.3	Tracking	5
3.3.1	Nearest-Neighbour Association	5
3.3.2	Track Lifecycle	6
3.3.3	Complexity	6
3.4	Classification	6
3.4.1	Motion Model	6
3.4.2	Ordinary Least Squares	7
3.4.3	Goodness of Fit: R^2	7
3.4.4	Classification Rule	7
4	Detailed Design and Implementation	7
4.1	Hardware	7
4.2	Firmware Architecture	8
4.3	Configuration Parameters	8
4.4	Communication Protocol	8
5	User's Manual	9
5.1	Prerequisites	9
5.2	Hardware Assembly	9
5.3	Configuration	10
5.4	Building and Flashing	10
5.5	Central Server	10
5.6	Troubleshooting	11
6	Conclusion	11

1 Introduction

1.1 Context

This project was motivated by the Building Automation course at TUCN (with prof. Valentin Sita). It addresses the problem of analysing the flow of people through a building by monitoring, in real time, how many people pass through each door and in which direction. Each door of interest is equipped with a small thermal sensor and a microcontroller - the *door unit* - which reports events (IN or OUT, with a timestamp) to a central unit over the network. The central unit can accumulate, plot, and process this data however needed.

From a technical standpoint the project sits at the intersection of edge computing and computer vision: a complete detection-tracking-classification pipeline runs locally on the microcontroller, so no raw sensor data ever leaves the device.

1.2 Specific Domain

This solution targets the hobbyist branch of building automation. Three closely related problems help situate it:

- **Named-individual tracking** (e.g. corporate login/logout systems): extracts identity; requires biometric hardware; high cost, high information content.
- **Undirected traffic monitoring** (e.g. counting visitors in a retail space): measures busyness, not flow; direction is not needed.
- **This project - directional anonymous counting**: detects entry and exit events without storing any identifying information, suitable for universities, churches, and similar public spaces where a login/logout system is disproportionate.

The output is a time series of the occupancy count: how many people are inside the building as a function of time.

2 Constraints and Requirements

The goal is to **obtain an accurate time series of occupancy** by monitoring every exit. The key constraints are:

Accuracy. Detection of *direction* is mandatory; robustness against simultaneous passages (multiple people at once) is desirable. This alone rules out the simpler dual IR beam-break approach, which cannot distinguish direction in an ambiguous crossing and fails entirely with multiple simultaneous people.

Cost. The solution must be price-efficient and require minimal maintenance. Reducing per-door hardware cost is essential when instrumenting every door in a building.

Privacy. No information that could identify an individual may be stored or transmitted. This rules out conventional camera-based approaches. The chosen sensor produces images of such low resolution that no individual can be identified; the processed output is a binary event (IN or OUT).

Power. The unit must operate continuously during any period when traffic may occur, potentially for extended durations without intervention.

Latency. Events should reach the central unit promptly so that the occupancy count is always close to real time. Batch uploads at end-of-day are not acceptable.

The proposed design satisfies all five: the thermal sensor at minimum usable resolution provides just enough spatial information for the algorithm to work while keeping compute requirements well within reach of an

inexpensive microcontroller; the low-resolution frames are inherently privacy-preserving; events are streamed immediately over the local network.

3 Analysis and Theoretical Foundation

3.1 System Overview

The door unit runs a four-stage pipeline, illustrated in Figure 1.

1. **Sensing**: the thermal sensor provides a low-resolution 2D frame of apparent temperatures at a fixed frame rate.
2. **Detection**: each frame is processed to locate the positions of heat sources (people) within it.
3. **Tracking**: detections across successive frames are linked into trajectories.
4. **Classification**: completed trajectories are analysed to determine direction (IN, OUT, or noise).



Figure 1: Pipeline overview.

3.2 Detection

Problem. Given a noisy 2D thermal frame, locate the spatial positions of heat sources (people).

We assume: the floor has a roughly uniform temperature; people appear as blobs significantly warmer than the background; and at most two people are in physical contact at any moment.

3.2.1 Background Estimation

The ambient temperature T_{amb} is estimated as the mean of the k coldest pixels in the frame. Using the global mean would be biased upward whenever hot blobs are present. The coldest pixels are almost always unoccupied floor, making the k -coldest mean a robust estimator of background temperature even during busy passages.

3.2.2 Thresholding

A pixel at grid position (i, j) is classified as *hot* if

$$T[i, j] > T_{\text{amb}} + \delta,$$

where $\delta > 0$ is a fixed offset chosen to exceed typical sensor noise. The result is a binary mask H over the frame.

3.2.3 Connected Component Labelling

Hot pixels that are spatially adjacent form a *blob*. We use 4-connectivity: two pixels belong to the same blob if and only if they share an edge (up, down, left, or right). Diagonal neighbours are not considered connected. This choice is conservative on a coarse grid: diagonal connectivity would join blobs that merely touch at a corner, leading to spurious merges.

The standard flood-fill algorithm labels all blobs in $O(n)$ time, where n is the number of pixels - a single pass suffices.

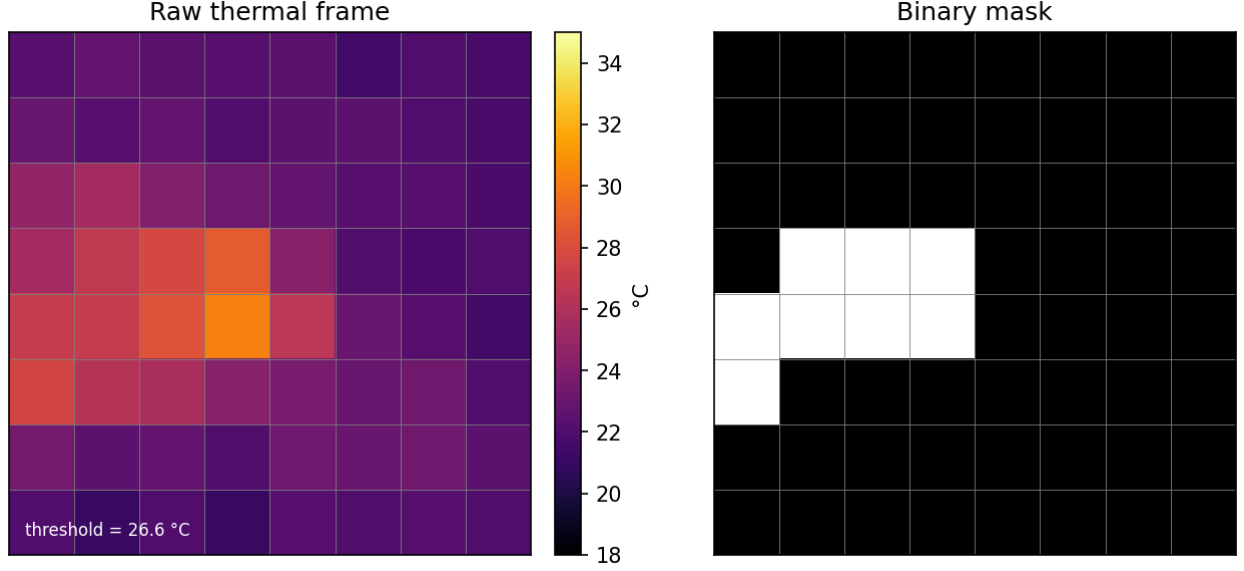


Figure 2: Raw thermal frame (left) and binary mask after thresholding (right).

3.2.4 Intensity-Weighted Centroid

For a blob with pixel set $\mathcal{B}$, each pixel being (r_k, c_k) , define the per-pixel weight $w_k = T[k] - T_{\text{threshold}}$ (the temperature excess above the threshold). The weighted centroid is

$$\bar{r} = \frac{\sum_{k \in \mathcal{B}} w_k r_k}{\sum_{k \in \mathcal{B}} w_k}, \quad \bar{c} = \frac{\sum_{k \in \mathcal{B}} w_k c_k}{\sum_{k \in \mathcal{B}} w_k}.$$

Weighting by excess temperature pulls the centroid toward the core of the body - the hottest region - rather than the cooler periphery of the blob, improving localisation accuracy.

3.2.5 Multi-Person Blob Splitting via PCA

Two people walking side by side may thermally merge into one large blob. If a blob is large enough to plausibly contain two people, we attempt to split it along its axis of greatest elongation using *Principal Component Analysis* (PCA).

PCA is a technique for finding the directions in which a set of points varies the most. Here we need only the first principal component (the direction of maximum variance), and we work in 2D, so the full machinery reduces to a simple eigenvalue problem on a 2×2 matrix.

Weighted covariance matrix. Let the blob contain N pixels with coordinates (r_k, c_k) and weights w_k . The 2×2 weighted covariance matrix is

$$\Sigma = \begin{pmatrix} C_{rr} & C_{rc} \\ C_{rc} & C_{cc} \end{pmatrix},$$

where

$$C_{rr} = \sum_k w_k (r_k - \bar{r})^2, \quad C_{cc} = \sum_k w_k (c_k - \bar{c})^2, \quad C_{rc} = \sum_k w_k (r_k - \bar{r})(c_k - \bar{c}).$$

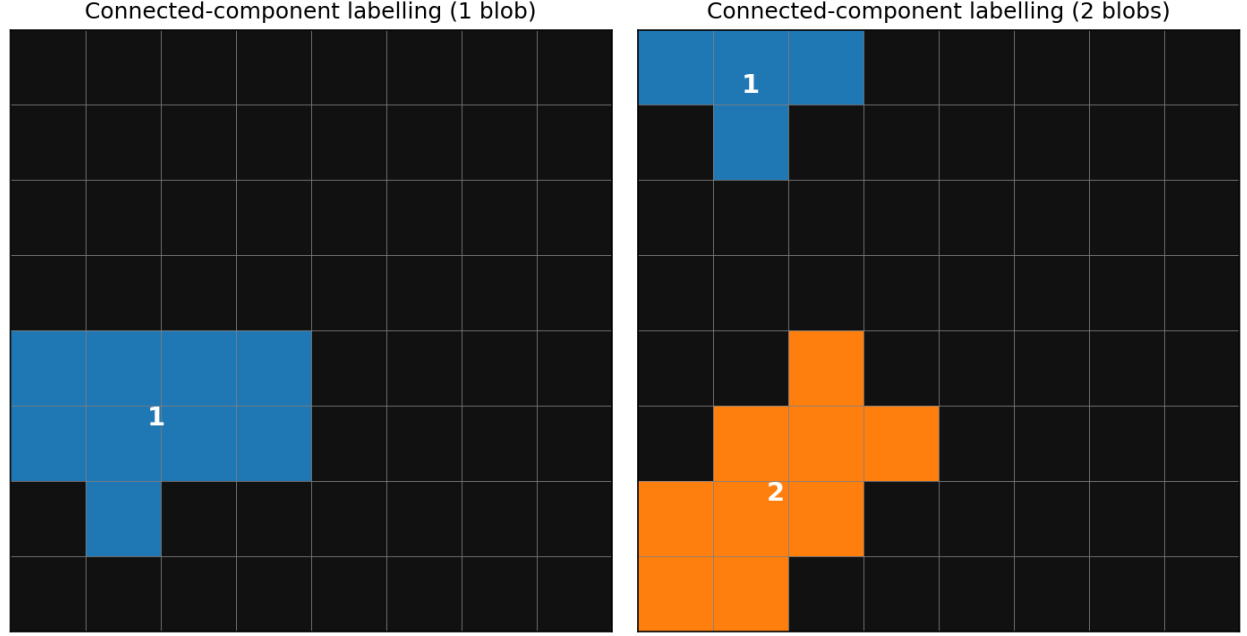


Figure 3: Connected component labelling (4-connectivity).

Intuitively, C_{rr} and C_{cc} measure how spread out the blob is along each axis, while C_{rc} captures the correlation between the two - i.e. whether the blob is tilted diagonally.

Principal axis. The principal axis is the eigenvector $\mathbf{v}$ of Σ associated with its largest eigenvalue $\lambda_{\max}$. For a 2×2 symmetric matrix the eigenvalues are

$$\lambda_{1,2} = \frac{(C_{rr} + C_{cc}) \pm \sqrt{(C_{rr} - C_{cc})^2 + 4C_{rc}^2}}{2}.$$

The corresponding eigenvector (normalised) is obtained directly from the matrix entries.

Split. Project each pixel onto the principal axis: $s_k = (r_k - \bar{r})v_r + (c_k - \bar{c})v_c$. Partition the pixels at the median projection s_{med} :

$$\mathcal{B}_1 = \{k : s_k \leq s_{\text{med}}\}, \quad \mathcal{B}_2 = \{k : s_k > s_{\text{med}}\}.$$

Compute one weighted centroid per partition. If the split fails to place any pixel in one partition (degenerate case), the blob is treated as a single detection.

3.3 Tracking

Problem. Given a set of detections at each frame, associate them with previously seen blobs so as to form continuous trajectories for each individual.

3.3.1 Nearest-Neighbour Association

At each frame we have a set of active tracks $\mathcal{T} = \{t_1, \dots, t_m\}$, each carrying its last known centroid, and a set of new detections $\mathcal{D} = \{d_1, \dots, d_n\}$. Association proceeds greedily: for each detection, find the active track whose last centroid is closest (Euclidean distance in pixel coordinates) and whose distance is below an acceptance threshold; assign the detection to that track. Each track can be assigned at most one detection per frame, and each detection to at most one track.

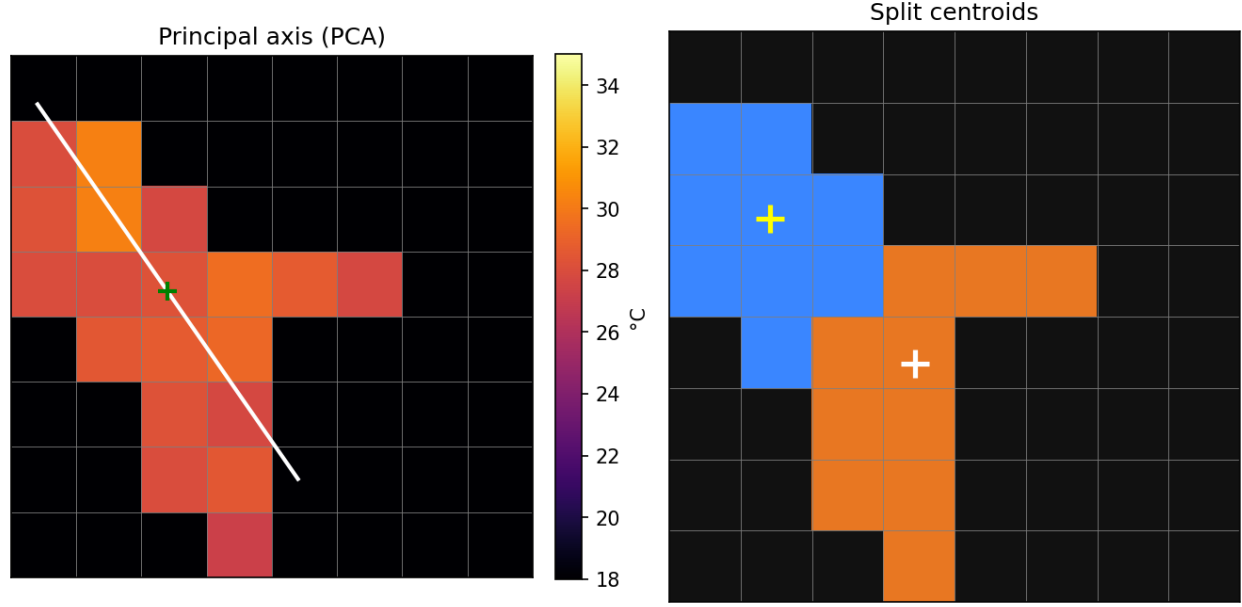


Figure 4: PCA-based blob splitting. The principal axis (left) separates the merged blob into two centroids (right).

Unmatched detections start new tracks. Unmatched tracks accumulate an *absence counter*; once the counter reaches a fixed limit the track is declared dead and passed to the classifier.

3.3.2 Track Lifecycle

Created when a detection has no sufficiently close active track.

Updated each frame it receives a matching detection; the new centroid is appended to the track's trajectory.

Terminated after too many consecutive frames without a match; the complete trajectory is forwarded to the classifier. This tolerance for short absences allows the algorithm to survive transient occlusions (e.g. two people briefly overlapping).

3.3.3 Complexity

With at most M tracks and N detections per frame - both bounded by the number of resolvable blobs in the frame - greedy nearest-neighbour matching runs in $O(MN)$. Since both M and N are small (bounded by the frame resolution), the cost is negligible.

3.4 Classification

Problem. Given a completed trajectory, determine whether it represents a person entering, exiting, or should be discarded as noise.

3.4.1 Motion Model

A person crossing a doorway moves approximately in a straight line along the axis perpendicular to the door. Let p_i denote the position of the tracked blob along this axis at frame i , for $i = 0, 1, \dots, N - 1$. We model this as a linear process with additive noise:

$$p_i = \beta \cdot i + \alpha + \varepsilon_i,$$

where β is the (signed) velocity, α is the initial position, and ε_i is zero-mean noise. The sign of β encodes direction.

3.4.2 Ordinary Least Squares

We estimate β and α by ordinary least squares (OLS). The standard closed-form solution is

$$\hat{\beta} = \frac{N \sum_i i p_i - (\sum_i i) (\sum_i p_i)}{N \sum_i i^2 - (\sum_i i)^2},$$

$$\hat{\alpha} = \frac{\sum_i p_i - \hat{\beta} \sum_i i}{N}.$$

The denominator simplifies because the frame indices are consecutive integers: $\sum_i i = N(N-1)/2$ and $\sum_i i^2 = N(N-1)(2N-1)/6$. No matrix inversion is needed.

The estimated total displacement over the trajectory is

$$\Delta p = \hat{\beta} \cdot (N-1).$$

3.4.3 Goodness of Fit: R^2

The coefficient of determination measures how well the linear model explains the data:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}, \quad SS_{\text{res}} = \sum_i (p_i - \hat{p}_i)^2, \quad SS_{\text{tot}} = \sum_i (p_i - \bar{p})^2,$$

where $\hat{p}_i = \hat{\beta}i + \hat{\alpha}$ is the fitted value and $\bar{p}$ is the mean position. $R^2 = 1$ indicates a perfect fit; $R^2 = 0$ means the model explains nothing beyond the mean. Values well below 1 correspond to curved or erratic trajectories - a person pausing at the door, two overlapping people, or sensor noise - which we want to discard.

3.4.4 Classification Rule

A trajectory is classified as a valid crossing only if:

1. It contains at least a minimum number of frames (the person must be visible for long enough),
2. The absolute displacement $|\Delta p|$ exceeds a minimum threshold (the person must actually cross the door, not merely approach it), and
3. R^2 exceeds a minimum threshold (the motion must be approximately linear).

Trajectories failing any criterion are classified as NOISE. Among the passing trajectories, the sign of Δp determines direction: by convention, positive displacement corresponds to entry (IN) and negative to exit (OUT), with the sign convention chosen to match the physical mounting of the sensor.

4 Detailed Design and Implementation

4.1 Hardware

Thermal sensor. The AMG8833 is an 8×8 thermopile array sensor communicating over I2C at up to 400kHz. It delivers frames at 1 or 10 Hz with a resolution of 0.25 °C per LSB. The 8×8 resolution was deliberately chosen: it is the minimum that gives the algorithm enough spatial information to work, while keeping both compute load and cost low. Crucially, it is insufficient to identify any individual.

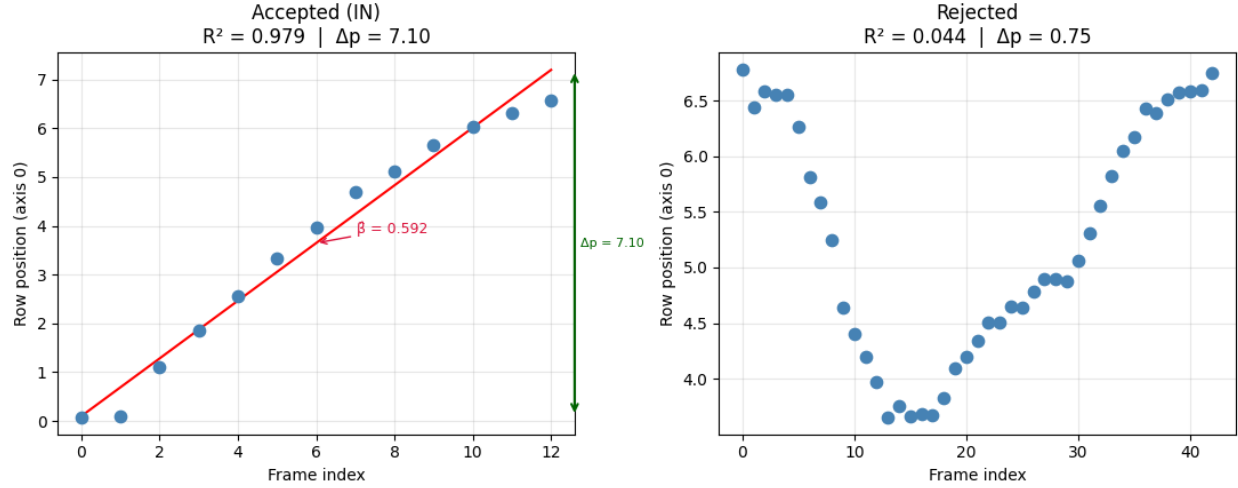


Figure 5: OLS regression on a trajectory (left: passing, right: rejected by R^2).

Microcontroller. The ESP32 is a dual-core 240 MHz Xtensa LX6 microcontroller with 520 KB SRAM and integrated 802.11 b/g/n Wi-Fi. It was chosen for its price, connectivity, and the comfortable headroom it provides over the algorithm’s compute and memory requirements.

Mounting geometry. The sensor is mounted face-down above the door frame, centred over the passage. With this orientation, a person walking through the door moves from one row to another; their column position changes only laterally. The algorithm tracks motion along the row axis to determine direction. Figure 6 shows the wiring and mounting arrangement.

4.2 Firmware Architecture

The firmware is built on ESP-IDF with FreeRTOS and consists of two independent tasks:

- **Pipeline task** (runs on core 1): reads a frame from the sensor every 100 ms, runs the full detection → tracking → classification pipeline, and enqueues any resulting events. Using `vTaskDelayUntil` ensures jitter-free 10 Hz execution even when processing time varies slightly.
- **Reporter task** (runs on core 0): drains the event queue and transmits events over TCP to the central server, reconnecting automatically on failure. Separating this task onto the Wi-Fi core prevents network I/O from introducing latency spikes in the pipeline.

All working buffers are statically allocated at compile time; there is no dynamic heap allocation in the hot path.

4.3 Configuration Parameters

Table 1 lists the tuneable constants and their default values.

4.4 Communication Protocol

Events are transmitted over TCP as newline-terminated ASCII messages:

[BOOT_]IN|OUT <timestamp_ms>

The `BOOT_` prefix is used when NTP time synchronisation has not yet completed, in which case the timestamp is milliseconds since boot rather than Unix time. After NTP sync the prefix is omitted and the timestamp is

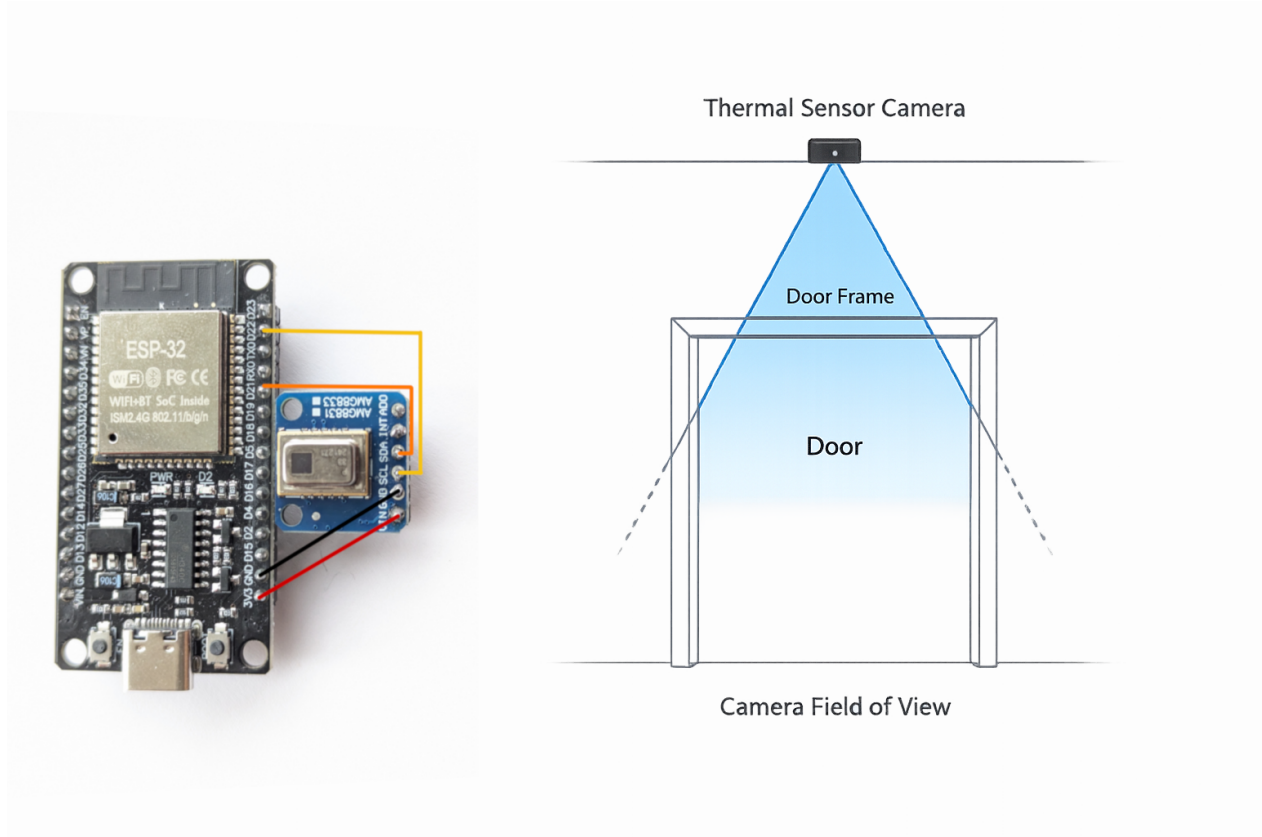


Figure 6: Left: sensor wiring; Right: camera location

a standard Unix millisecond value. A non-blocking FreeRTOS queue (depth 16) decouples event production from TCP I/O; if the server is unreachable, events are dropped after the queue fills.

5 User's Manual

5.1 Prerequisites

- ESP-IDF v5.x installed with `idf.py` on the system path
- AMG8833 breakout board (with 3.3 V regulator and pull-up resistors, as found on standard modules)
- ESP32 development board with USB-UART programmer
- USB cable

5.2 Hardware Assembly

Connect the AMG8833 to the ESP32 as follows:

AMG8833 pin	ESP32 pin
VCC	3V3
GND	GND
SDA	GPIO 21
SCL	GPIO 22

Table 1: Key configuration parameters (`main/config.h`).

Parameter	Default	Role
TEMP_OFFSET	5.0 °C	Threshold offset above ambient
MIN_HOT_PIXELS	4	Minimum blob size (pixels)
SPLIT_THRESHOLD	16	Blob size above which PCA split is attempted
MAX_MATCH_DIST	3.0 px	Maximum centroid distance for track association
ABSENT_FRAMES_MAX	3	Absent frames before a track is terminated
MIN_TRAJ_FRAMES	3	Minimum trajectory length for classification
MIN_DISPLACEMENT	3.0 px	Minimum $ \Delta p $ for a valid crossing
MIN_R_SQUARED	0.6	Minimum R^2 for a valid crossing
GOOD_AXIS	0 (row)	Axis along which direction is measured
IN_DIRECTION	+1	Sign of displacement corresponding to entry

I2C address: The AMG8833 address is set by the AD0 pin. AD0 low gives address 0x68; AD0 high gives 0x69. Most breakout boards pull AD0 high by default. The firmware uses 0x69; change `AMG_I2C_ADDR` in `config.h` if needed.

Mounting: Mount the sensor face-down, centred above the door frame. The sensor’s rows should run perpendicular to the door opening, so that a person walking through moves from one row to the next. Verify that the field of view covers the full width of the passage.

5.3 Configuration

Open `main/config.h` and set:

- `WIFI_SSID` and `WIFI_PASS` - your network credentials.
- `SERVER_HOST` and `SERVER_PORT` - IP address and port of the central server.
- `GOOD_AXIS` and `IN_DIRECTION` - verify these match your mounting orientation (see Section 3.2.5 and Table 1). The defaults assume rows run perpendicular to the door and increasing row index corresponds to entry.
- Optionally adjust `TEMP_OFFSET` for your environment. Colder rooms may need a lower value; rooms with underfloor heating may need a higher one.

5.4 Building and Flashing

```
idf.py set-target esp32
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

On a successful boot the serial monitor should show: Wi-Fi connected, IP address acquired, SNTP started, sensor found on I2C, and pipeline running.

5.5 Central Server

The receiver side is currently a stub. For testing, `netcat` is sufficient:

```
nc -l 4444 | tee events.log
```

A production receiver should:

- Accept TCP connections and handle reconnections gracefully (the door unit reconnects every 3 seconds on failure).

- Parse lines of the form [BOOT_] IN|OUT <timestamp_ms> and maintain a running count.
- Correlate BOOT_-prefixed timestamps with wall-clock time if precise timing is needed.

5.6 Troubleshooting

Sensor not found on boot. Check I2C wiring. Verify the I2C address (0x68 vs. 0x69) by checking the AD0 pin or using an I2C scanner sketch.

No detections. The environment temperature may be too close to body temperature. Lower TEMP_OFFSET.

Too many false detections (hot objects, sunlight patches). Raise TEMP_OFFSET, or consider shielding the sensor from direct heat sources.

False IN/OUT events (noise, brief approaches). Raise MIN_DISPLACEMENT or MIN_R_SQUARED.

Missed crossings. Lower MIN_DISPLACEMENT. Check that the sensor FOV covers the full door width; a person walking along one edge may be too small to trigger.

6 Conclusion

This project demonstrates that a complete, privacy-preserving, directional people counting system can be built from inexpensive off-the-shelf components. The pipeline - background-robust thresholding, connected component detection with PCA-based splitting, greedy nearest-neighbour tracking, and linear regression classification - runs entirely on the microcontroller, producing only a stream of timestamped IN/OUT events.

Current limitations include the absence of a real receiver implementation, no local event buffering when the network is unavailable, and reduced accuracy when more than two people pass simultaneously. On the algorithmic side, the 8×8 thermal frame contains more information than the current pipeline exploits. Note that the code is organized and supports future expansion, refinement and swapping components (like the tracker, the classifier) with better ones.

Natural directions for future development are: a proper receiver with data storage and visualisation; extending connectivity beyond the local network (e.g. reporting to a mobile endpoint); and improving the detection stage to better handle crowded passages, especially at higher sensor mounting heights.